Informe técnico del proyecto: Registro de Estudiantes

Tipo de sistema: Aplicación web CRUD para administrar estudiantes

Tecnologías identificadas: Python 3, Flask, MongoDB/PyMongo, HTML/Jinja2 y CSS

Material analizado: `registro_estudiantes.zip` (archivo ZIP, no RAR)

Fecha de elaboración: 30 de octubre de 2026

1. Resumen ejecutivo

El proyecto implementa un sitio web para administrar un registro de estudiantes. Permite crear fichas con nombre, apellido y correo electrónico; consultar el listado y el detalle individual; buscar por nombre o apellido; editar los datos y eliminar registros. La aplicación usa Flask para atender las solicitudes web, clases propias para representar y validar los datos, y una capa repositorio para separar la lógica del mecanismo de almacenamiento. En el uso normal, los datos se guardan en MongoDB. Existe además un repositorio temporal en memoria pensado para probar la interfaz sin una base de datos.

La estructura es modular y se distinguen las capas de presentación, aplicación, dominio y persistencia. La información de cada estudiante se limita a tres campos personales, además de un identificador generado por el almacenamiento. El código valida los campos en el servidor y también declara restricciones básicas en el formulario HTML.

2. Alcance y estructura del paquete

El archivo recibido es un ZIP que contiene una carpeta `registro_estudiantes/`. Dentro hay tres archivos Python, un archivo de dependencias, instrucciones de instalación, una hoja CSS y cinco plantillas HTML. No se encontraron otros archivos de programa, scripts JavaScript, pruebas automatizadas, archivos de configuración de despliegue ni esquemas SQL.

Ruta	Tipo	Propósito
`app.py`	Python	Inicializa Flask, configura el repositorio y define las rutas HTTP y sus respuestas.
`modelos.py`	Python	Define las entidades `Estudiante` y `ListaEstudiantes`, con validación y reglas del negocio.
`repositorio.py`	Python	Implementa almacenamiento MongoDB y alternativa temporal en memoria.
`requirements.txt`	Dependencias	Declara Flask y PyMongo como paquetes requeridos.
`INSTALACION.md`	Documentación	Explica los requisitos, instalación, ejecución y modo de prueba.
`static/estilos.css`	CSS	Define colores, distribución, componentes y adaptación a pantallas pequeñas.
`templates/base.html`	Jinja2/HTML	Plantilla común con navegación, mensajes y estructura general.
`templates/inicio.html`	Jinja2/HTML	Página de bienvenida y contador de estudiantes.
`templates/lista.html`	Jinja2/HTML	Listado, búsqueda y accesos a ver, editar y eliminar.
`templates/formulario.html`	Jinja2/HTML	Formulario reutilizado para crear o editar una ficha.

Ruta	Tipo	Propósito
`templates/detalle.html`	Jinja2/HTML	Presenta los datos de una persona y las acciones disponibles.
`templates/error.html`	Jinja2/HTML	Presenta errores en una pantalla amigable.

Las carpetas `static/` y `templates/` son directorios contenedores, no programas por sí mismos. Los archivos HTML usan Jinja2, el motor de plantillas que Flask integra.

3. Descripción funcional para el usuario

1. **Inicio:** presenta un mensaje de bienvenida, cuenta el total de fichas y ofrece accesos para agregar una persona o ver el listado.

2. **Alta:** el formulario solicita nombre, apellido y email. Al enviar, el servidor valida la información, guarda el registro y vuelve al listado con un mensaje de éxito. Si la validación falla, se muestra el formulario con los valores ingresados y el error.

3. **Consulta y búsqueda:** el listado muestra apellido, nombre y email ordenados alfabéticamente por apellido y nombre. El buscador filtra por coincidencia en nombre o apellido sin distinguir mayúsculas y minúsculas.

4. **Detalle:** se muestran nombre completo, nombre, apellido y correo; el email es un enlace `mailto:`.

5. **Edición:** el formulario se precarga con los valores existentes. Tras guardar, se actualiza el registro y se redirige al detalle.

6. **Eliminación:** el usuario confirma la acción en el navegador. El formulario envía una petición POST; el servidor intenta borrar el registro y vuelve al listado mostrando el resultado.

4. Arquitectura y recorrido de datos

La interfaz del navegador envía una solicitud a una ruta Flask de `app.py`. Para una operación de negocio, Flask llama a `ListaEstudiantes` en `modelos.py`. Esta capa construye o recupera objetos `Estudiante`, ejecuta validaciones y delega la persistencia al repositorio. El repositorio usa PyMongo para consultar MongoDB o un diccionario Python si se activó el modo memoria. El resultado vuelve a Flask, que renderiza una plantilla Jinja2; el CSS de `static/estilos.css` controla su aspecto.

Capas: presentación (HTML/Jinja2 y CSS) → aplicación/controlador (`app.py`) → dominio y reglas (`modelos.py`) → almacenamiento (`repositorio.py`) → MongoDB.

La separación mediante contrato de repositorio permite cambiar o sustituir el almacenamiento sin reescribir las reglas del negocio ni las rutas principales. La colección MongoDB se llama `estudiantes` dentro de la base `registro_alumnos` por defecto. Cada documento contiene `_id`, `nombre`, `apellido` y `email`.

5. Explicación detallada de cada archivo de programa

5.1 `app.py` — aplicación web y controladores

Importa Flask y funciones para mostrar plantillas, leer formularios, redirigir y emitir mensajes temporales. Importa `ListaEstudiantes` y los dos repositorios. Crea la aplicación Flask y configura `secret_key`, necesaria para los mensajes flash. La clave usa una cadena por defecto que el propio código indica que debe cambiarse en producción.

Selecciona `RepositorioMemoria` si la variable de entorno `USAR_MEMORIA` vale `1`; en caso contrario instancia `RepositorioEstudiantes`, usando `MONGO_URI` y `MONGO_BD`, con valores predeterminados de Mongo local y `registro_alumnos`. Después crea el servicio `lista`, que será utilizado por las rutas.

La función auxiliar `_datos_form()` extrae `nombre`, `apellido` y `email` del formulario POST. El manejador global `error_general()` muestra errores HTTP con su descripción y código; para errores no HTTP registra la excepción en el log y presenta un mensaje genérico indicando que podría fallar MongoDB. La aplicación devuelve estado 500 en este caso.

Ruta	Método	Función
`/`	GET	`inicio()` cuenta registros y renderiza `inicio.html`.
`/estudiantes`	GET	`listar()` lee el parámetro `q`, busca coincidencias y renderiza `lista.html`.
`/estudiantes/nuevo`	GET/POST	`nuevo()` muestra el formulario o valida/guarda un estudiante nuevo.
`/estudiantes/<id>`	GET	`detalle(id)` muestra la ficha; si no existe, informa y vuelve al listado.
`/estudiantes/<id>/editar`	GET/POST	`editar(id)` muestra los datos o actualiza la ficha tras validar.
`/estudiantes/<id>/eliminar`	POST	`eliminar(id)` borra la ficha si existe y redirige al listado.

Cuando una operación de alta o edición produce `ValueError`, `app.py` presenta el mensaje de validación con `flash` y vuelve a renderizar el formulario, conservando los valores enviados. Cuando el alta funciona, redirige al listado; cuando la edición funciona, redirige al detalle. El bloque final ejecuta el servidor de desarrollo con `debug=True` si se inicia directamente `python app.py`.

5.2 `modelos.py` — entidades, validación y lógica del negocio

Compila una expresión regular simple para verificar un formato de correo similar a `algo@dominio.ext`.

Clase `Estudiante`: representa una ficha individual. Guarda los atributos internos de manera privada (`__id`, `__nombre`, `__apellido`, `__email`) y expone propiedades. Los setters normalizan y validan: nombre y apellido eliminan espacios externos, exigen al menos dos caracteres y convierten las palabras a formato de título; email elimina espacios externos, pasa a minúsculas y debe coincidir con el patrón. Si un dato no cumple, lanzan `ValueError`. El identificador se asigna desde el repositorio. `nombre_completo` concatena nombre y apellido; `a_diccionario()` produce los campos persistibles sin `_id`; `desde_diccionario()` reconstruye un objeto desde un documento de MongoDB.

Clase `ListaEstudiantes`: coordina las operaciones de negocio y recibe un repositorio por inyección en su constructor. `agregar()` construye una entidad validada y la guarda; `editar()` recupera la entidad, comprueba que exista, asigna de nuevo los campos (lo cual vuelve a validarlos) y solicita su actualización; `eliminar()` delega el borrado; `obtener()` y `obtener_todos()` delegan consultas; `buscar_por_nombre()` solicita una búsqueda filtrada si el texto no está vacío; `contar()` obtiene la cantidad a partir de la lista recuperada.

5.3 `repositorio.py` — persistencia

Documenta un contrato común: `guardar`, `cargar`, `obtener`, `actualizar` y `eliminar`. Así `ListaEstudiantes` puede trabajar con cualquiera de las implementaciones sin conocer detalles de MongoDB.

`RepositorioEstudiantes`: conecta con MongoDB utilizando `MongoClient` y PyMongo; usa timeout de selección de servidor de 3 segundos. Accede a la colección `estudiantes`. `guardar()` inserta el diccionario y devuelve el ObjectId como texto. `cargar()` recupera documentos, opcionalmente aplicando una expresión regular escapada (`re.escape`) en nombre o apellido, sin distinguir mayúsculas, y ordena por apellido y nombre. `obtener()` convierte el identificador textual a `ObjectId` de forma segura y devuelve la entidad si existe. `actualizar()` usa `$set` para modificar los campos sin reemplazar todo el documento. `eliminar()` ejecuta el borrado y retorna verdadero solo si se eliminó un documento. Un ID mal formado se convierte en ausencia de resultado y no provoca un error de conversión.

`RepositorioMemoria`**:** almacena diccionarios dentro de un diccionario privado, asignando identificadores UUID. Implementa el mismo contrato: agrega, lista/filtra/ordena, recupera, actualiza y elimina. Su contenido vive únicamente en memoria del proceso, por eso se pierde al cerrar o reiniciar la aplicación. Es útil para mostrar y probar la interfaz sin MongoDB, pero no es almacenamiento permanente.

5.4 `requirements.txt` — dependencias

Declara `Flask>=3.0` para servidor y plantillas, y `pymongo>=4.6` para comunicación con MongoDB. La versión de Python indicada en la guía de instalación es 3.10 o superior. Las dependencias se instalan con `pip install -r requirements.txt`.

5.5 `INSTALACION.md` — instrucciones de puesta en marcha

Pide Python 3.10+ y MongoDB (local, Compass/Atlas como opciones citadas). Indica crear y activar un entorno virtual, instalar dependencias y ejecutar `python app.py`. Permite ajustar la conexión con `MONGO_URI` y la base con `MONGO_BD`. Para modo sin MongoDB se debe definir `USAR_MEMORIA=1` (en Windows, `set`; en Linux/macOS, `export`) antes de ejecutar. Finalmente indica abrir `http://127.0.0.1:5000`.

5.6 `static/estilos.css` — presentación visual

Define variables de color, estilos globales, barra azul de navegación, contenedor centrado y tarjetas blancas. Estiliza botones normales, secundarios y de peligro; mensajes de éxito/error; formularios; tabla; buscador y estado vacío. La regla `box-sizing: border-box` facilita el cálculo de dimensiones. Una consulta de medios para anchos menores de 640 px oculta encabezados de la tabla, presenta cada fila como bloque, agrega etiquetas mediante `data-label` y permite que el buscador se ajuste en varias líneas. No se encontró JavaScript separado; la confirmación de borrado se implementa como atributo inline en los formularios HTML.

6. Explicación de las plantillas HTML

Las cinco plantillas extienden `base.html` salvo la base misma. Los bloques `{% block %}` permiten sustituir título y contenido; expresiones `{{ ... }}` insertan valores de servidor y las estructuras `{% if %}` / `{% for %}` deciden qué mostrar o recorren resultados.

- `base.html`**:** declara idioma español, codificación UTF-8, diseño adaptable a viewport y título predeterminado. Vincula el CSS usando `url_for`, incluye cabecera con enlace al inicio y navegación a listado y alta. Recupera mensajes flash con sus categorías para mostrarlos como avisos y ofrece un bloque principal para cada página.

- `inicio.html`**:** muestra bienvenida, texto descriptivo, número total registrado y dos botones de navegación. Cambia las palabras a singular/plural según el total.

- `lista.html`**:** contiene buscador GET con el parámetro q; si hay búsqueda ofrece enlace para limpiarla. Cuando hay resultados crea una tabla con apellido, nombre y email, y acciones de ver/editar/eliminar. Cuando no hay resultados presenta un mensaje apropiado. Cada fila usa `data-label` para la vista móvil.

- `formulario.html`**:** sirve tanto para alta como edición según el título enviado por Flask. Incluye nombre, apellido y correo, atributos `required`, longitud mínima 2 para nombre/apellido y tipo email para el navegador. Conserva datos con `datos.get(...)`; ofrece guardar y cancelar. La validación cliente mejora usabilidad, mientras que la validación confiable se repite en Python.

- `detalle.html`**:** muestra el nombre completo y campos; el correo usa `mailto:`. Tiene enlaces para editar y volver, además de un formulario POST para eliminar. El `confirm()` del navegador pide confirmación antes de enviar.

- `error.html`**:** muestra el encabezado “Ups, algo salió mal”, el mensaje recibido y un enlace al inicio.

7. Validaciones y comportamiento de datos

El servidor es la autoridad de validación. No permite nombres o apellidos con menos de dos caracteres después de quitar espacios. Normaliza nombres y apellidos con `.title()` y el correo con `.lower()`. El patrón de email es deliberadamente básico: descarta espacios, exige una arroba, texto antes y después, un punto y texto final; no verifica que el dominio exista ni que la dirección pueda recibir mensajes. El formulario HTML añade validaciones para el navegador, que pueden omitirse por clientes no navegador, razón por la que las validaciones Python son necesarias.

La búsqueda solo contempla nombre y apellido; no busca por email. La coincidencia parcial se realiza mediante regex y se escapa el texto para que símbolos ingresados por el usuario se interpreten literalmente. El listado queda ordenado alfabéticamente.

8. Ejecución resumida

1. Instalar Python 3.10 o superior y MongoDB, salvo que se vaya a probar en memoria.

2. Crear/activar un entorno virtual e instalar `requirements.txt`.

3. Configurar, si hace falta, `MONGO_URI` y `MONGO_BD`, o exportar `USAR_MEMORIA=1` para la prueba temporal.

4. Ejecutar `python app.py` y navegar a `http://127.0.0.1:5000`.

5. Para un uso real, configurar una clave secreta segura y no ejecutar el servidor con modo debug expuesto.

9. Observaciones, límites y mejoras sugeridas

- **Clave secreta:** el valor por defecto `cambiar-en-produccion` no debe utilizarse en un despliegue real; hay que definir `SECRET_KEY` como secreto aleatorio fuera del código.

- **Modo debug:** `app.run(debug=True)` es apropiado para desarrollo local, no para producción. Se recomienda servidor WSGI y `debug=False`.

- **Protección CSRF:** los formularios POST no muestran integración de protección CSRF. Para uso accesible desde Internet conviene incorporar Flask-WTF/CSRF u otro mecanismo.

- **Integridad de datos:** no hay restricción de email único. Si se requiere que cada estudiante tenga correo distinto, deben añadirse validación de duplicados y un índice único en MongoDB.

- **Campos del modelo:** actualmente solo registra nombre, apellido y email. No incluye matrícula, curso, fecha de nacimiento, teléfono u otros datos, ni roles/autenticación de usuarios.

- **Pruebas:** no se incluyen pruebas automatizadas. Conviene probar altas válidas e inválidas, edición, búsqueda, IDs inexistentes, borrado y fallas de conexión.

- **Manejo de excepciones:** `editar()` puede levantar `LookupError` si el elemento deja de existir entre lectura y actualización; el manejador general lo convertiría en error 500. Podría manejarse como 404 o redirección controlada.

- **Contador:** `contar()` carga todos los estudiantes y cuenta en Python, lo cual puede ser ineficiente con muchos registros. MongoDB puede contar documentos directamente.

- **Disponibilidad de MongoDB:** el repositorio realiza una conexión diferida; un problema puede aflorar al ejecutar consultas y se presentará mediante el manejador general.

- **Privacidad:** los datos personales requieren acceso restringido, copias de seguridad y reglas de retención si el sistema se usa con datos reales.

Estas observaciones no impiden entender ni probar el proyecto; distinguen el alcance actual de requisitos habituales para producción.

10. Conclusión

El paquete contiene una aplicación web funcional de registro estudiantil con operaciones CRUD completas: crear, leer, actualizar y eliminar. Su principal fortaleza académica es la separación entre interfaz, reglas de dominio y persistencia, además del uso de encapsulación, propiedades, validación y un repositorio intercambiable. Para una demostración local puede funcionar con MongoDB o con almacenamiento en memoria. Antes de desplegarla para uso real, debería configurarse seguridad básica, pruebas automatizadas, protección CSRF, clave secreta segura y controles acordes a la privacidad de la información.

Glosario breve

CRUD: crear, leer, actualizar y eliminar. **Ruta/endpoint:** dirección que recibe solicitudes HTTP. **Jinja2:** motor de plantillas HTML de Flask. **Repositorio:** componente que aísla la forma de guardar y recuperar datos. **ObjectId:** identificador usado por MongoDB. **POST:** método HTTP usado aquí para enviar formularios que modifican datos. **Flash:** mensaje temporal que se muestra luego de una operación.